CPU Scheduling

CPU scheduling is the basis of multiprogrammed operating systems.

By switching the CPU among processes, the operating system can make the computer more productive.

- In a single-processor system, only one process can run at a time.
- Any others must wait until the CPU is free and can be rescheduled.

- The objective of multiprogramming is to have some process running at all times, to maximize CPU utilization.
- A process is executed until it must wait, typically for the completion of some I/O request.

In a simple computer system, the CPU then just sits idle. All this waiting time is wasted; no useful work is accomplished.

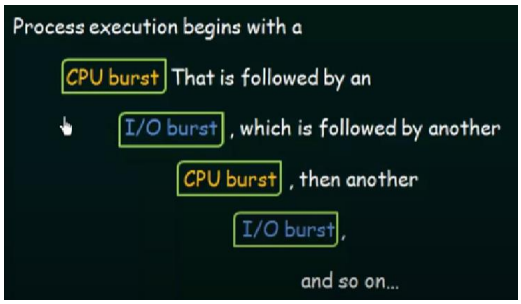
- With multiprogramming, we try to use this time productively.
- Several processes are kept in memory at one time.

- When one process has to wait, the operating system takes the CPU away from that process and gives the CPU to another process and this pattern continues.

CPU & I/O Burst Cycle

Process execution consists of a cycle of CPU execution & I/O wait.

Process alternate b/w these two process.

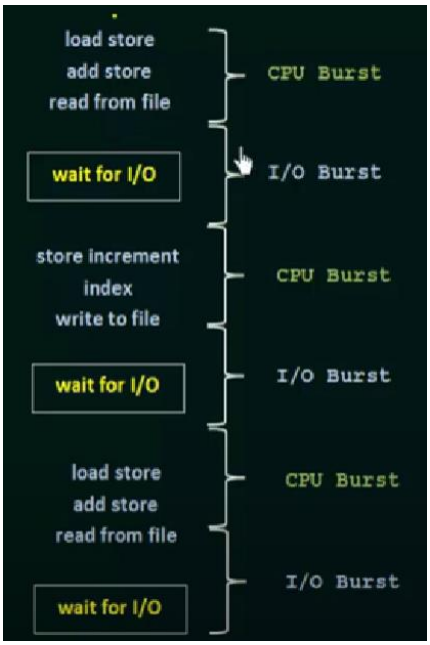


CPU burst is when the process is being executed in the CPU.

I/O burst is when the CPU is waiting for I/O for further execution.

Eventually the final CPU Burst end with a system request to terminate execution.

Alternate sequence of CPU Burst & I/O burst.



Pre-emptive and Non-Pre-emptive Scheduling

CPU Scheduler

Whenever the CPU becomes idle, the operating system must select one of the processes in the ready queue to be executed. The selection process is carried out by the short-term scheduler (or CPU scheduler). The scheduler selects a process from the processes in memory that are ready to execute and allocates the CPU to that process.

Dispatcher

The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler. The time it takes for the dispatcher to stop one process and start another running is known as the dispatch latency.

CPU-scheduling decisions may take place under the following four circumstances:

1. When a process switches from the running state to the waiting state.
2. 2. When a process switches from the running state to the ready state (for example, when an interrupt occurs).
3. 3. When a process switches from the waiting state to the ready state (for example, at completion of I/O).
4. When a process terminates.

For situations 1 and 4, there is no choice in terms of scheduling. A new process (if one exists in the ready queue) must be selected for execution. However, there is a choice for situations 2 and 3.

When scheduling takes place only under circumstances 1 & 4, we say that the scheduling scheme is non pre-emptive or cooperative, otherwise, it is pre-emptive.

Scheduling Criteria

1. CPU Utilization
2. Throughput
3. Turnaround rime
4. Waiting Time
5. Response Time

1) CPU Utilization

We want to keep the CPU as busy as possible. Conceptually, CPU utilization can range from 0 to 100 percent. In a real system, it should range from 40 percent (for a lightly loaded system) to 90 percent (for a heavily used system).

2) Throughput

If the CPU is busy executing processes, then work is being done. One measure of work is the number of processes that are completed per time unit, called throughput.

3) Turnaround rime

From the point of view of a particular process, the important criterion is how long it takes to execute that process. The interval from the time of submission of a process to the time of completion is the turnaround time. Turnaround time is the sum of the periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/O.

4) Waiting Time

The CPU scheduling algorithm does not affect the amount of time during which a process executes or does I/O; it affects only the amount of time that a process spends waiting in the ready queue. Waiting time is the sum of the periods spent waiting in the ready queue.

5) Response Time

In an interactive system, turnaround time may not be the best criterion. Often, a process can produce some output fairly early and can continue computing new results while previous results are being output to the user. Thus, another measure is the time from the submission of a request until the first response is produced. This measure, called response time, is the time it takes to start responding, not the time it takes to output the response. The turnaround time is generally limited by the speed of the output device.

Scheduling Algorithms

First Come First Served Scheduling

- By far the simplest CPU-scheduling algorithm.
- The process that requests the CPU first is allocated the CPU first.
- The implementation of the FCFS policy is easily managed with a FIFO queue.

- When a process enters the ready queue, its PCB is linked onto the tail of the queue.
- When the CPU is free it is allocated to the process at the head of the queue.
- The running process is then removed from the queue.

The average waiting time under the FCFS policy, however, is often quite long.

Consider the following set of processes that arrive at time 0.

Process	Burst Time(ms)
P1	24
P2	3
P3	3

If the processes arrive in the order P1, P2, P3, and are served in FCFS order, we get the result shown in the following Gantt chart:

P1	P2	P3

0 24 27 30

Waiting Time for P1 = 0 ms

Waiting Time for P2 = 24 ms

Waiting Time for P3 = 27 ms

Average Waiting Time = (0 + 24 + 27)/3 = 17 ms

If the processes arrive in the order P2, P3, P1, however the result will be shown in the following Gantt chart:

P2	P3	P1

0 3 6 30

Waiting Time for P1 = 6 ms

Waiting Time for P2 = 0 ms

Waiting Time for P3 = 3 ms

Average Waiting Time = (6 + 0 + 3)/3 = 3 ms

This reduction is substantial. Thus, the average waiting time under an FCFS policy is generally not minimal and may vary substantially if the process's CPU burst times vary greatly.

The FCFS algorithm is non-pre-emptive

• Once the CPU has been allocated to a process, that process keeps the CPU until it releases the CPU, either by terminating or by requesting I/o.

• The FCFS algorithm is thus particularly troublesome for time-sharing systems, where it is important that each user get a share of the CPU at regular intervals.

• It would be disastrous to allow one process to keep the CPU for an extended period.

Convoy Effect

If processes with higher burst time arrived before the processes with smaller burst time, then, smaller processes have to wait for a long time for longer processes to release the CPU.

Another Example:

Consider the set of 5 processes whose arrival time and burst time are given below:

Process ID	Arrival Time	Burst Time
P1	4	5
P2	6	4
P3	0	3
P4	6	2
P5	5	4

Calculate the average waiting time and average turnaround time, if FCFS Scheduling Algorithm is followed.

Solution:

Gantt Chart:

P3	//////////////////	P1	P5	P2	P4

0 3 4 9 13 17 19

Shaded box represents idle time of CPU.

Turnaround Time = Exit Time - Arrival Time

Waiting Time = Turnaround Time - Burst Time

Process ID	Arrival Time	Burst Time	Exit Time	Turnaround time	Waiting Time
P1	4	5	9	5	0
P2	6	4	17	11	7
P3	0	3	3	3	0
P4	6	2	19	13	11
P5	5	4	13	8	14

Now,

Average Turn Around time = (5 + 11 + 3 + 13 + 8) / 5

= 40 / 5

= 8 units

Average waiting time = (0 + 7 + 0 + 11 + 4) / 5

= 22 / 5

= 4.4 units

Shortest Job First Scheduling

• This algorithm associates with each process the length of the process's next CPU burst.

• When the CPU is available, it is assigned to the process that has the smallest next CPU burst.

• If the next CPU bursts of two processes are the same, FCFS scheduling is used to break the tie.

The SJF algorithm can be either pre-emptive or non-pre-emptive

A more appropriate term for this scheduling method would be the

Shortest-Next-CPU-Burst Algorithm

because scheduling depends on the length of the next CPU burst of a process, rather than its total length.

Example of SJF Scheduling (Non-Pre-emptive)

Consider the following set of processes, with the length of the CPU burst given in milliseconds:

Process ID	Burst Time
P1	6
P2	8
P3	7
P4	3

P4	P1	P3	P2

0 6 9 16 24

Waiting Time for P1 = 3 ms

Waiting Time for P2 = 16 ms

Waiting Time for P3 = 9 ms

Waiting Time for P4 = 0 ms

Average Waiting Time = (3 + 16 + 9 + 0)/4

= 7 ms

By comparison, if we were using the FCFS scheduling scheme, the average waiting time would be 10.25 ms.

Example of SJF Scheduling (Pre-emptive)

Consider the following four processes, with the length of the CPU burst given in milliseconds and the processes arrive at the ready queue at the times shown:

Process ID	Arrival Time	Burst Time
P1	0	~~8~~ 7
P2	1	4
P3	2	9
P4	3	5

Gantt Chart:

P1	P2	P4	P1	P3

0 1 5 10 17 26

Waiting Time = Total Waiting time -No. of ms process executed -Arrival Time

Waiting Time for P1 = (10 - 1 - 0) = 9 ms

Waiting Time for P2 = (1 - 0 - 1) = 0 ms

Waiting Time for P3 = (17 - 0 - 2) = 15 ms

Waiting Time for P4 = (5 - 0 - 3) = 2 ms

Average Waiting Time

= (9 + 0 + 15 + 2)/4

= 6.5 ms

Pre-emptive SJF Scheduling is sometimes called **Shortest-Remaining-Time-Scheduling**

Problems with SJF Scheduling:

• The real difficulty with the SJF algorithm is knowing the length of the next CPU request.

• Although the SJF algorithm is optimal, it cannot be implemented at the level of short-term CPU scheduling.

• There is no way to know the length of the next CPU burst.

One approach is:

• To try to approximate SJF scheduling.

• We may not know the length of the next CPU burst, but we may be able to predict its value.

• We expect that the next CPU burst will be similar in length to the previous ones.

• Thus, by computing an approximation of the length of the next CPU burst, we can pick the process with the shortest predicted CPU burst.

Priority Scheduling

• A priority is associated with each process, and the CPU is allocated to the process with the highest priority.

• Equal-priority processes are scheduled in FCFS order.

•An SJF algorithm is simply a priority algorithm where the priority is the inverse of the (predicted) next CPU burst.

The larger the CPU burst, the lower the priority, and vice versa.

Priority Scheduling can be either pre-emptive or non-pre-emptive

A pre-emptive priority scheduling algorithm will pre-empt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process.

A non pre-emptive priority scheduling algorithm will simply put the new process at the head of the ready queue.

Consider the following set of processes, assumed to have arrived at time 0, in the order P1, P2, P3, P4, P5, with the length of the CPU burst given in milliseconds:

Process ID	Burst Time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Using Priority Scheduling, we would schedule these processes according to the following Gantt Chart:

P2	P5	P1	P3	P4

0 1 6 16 18 19

Waiting Time for P1 = 6 ms

Waiting Time for P2 = 0 ms

Waiting Time for P3 = 16 ms

Waiting Time for P4 = 18 ms

Waiting Time for P5 = 1 ms

Average Waiting Time

= (6 + 0 + 16 + 18 + 1) / 5

= 41 / 5 ms

= 8.2 ms

Problem with Priority Scheduling:

• A major problem with priority scheduling algorithms is indefinite blocking, or starvation.

• A process that is ready to run but waiting for the CPU can be considered blocked.

• A priority scheduling algorithm can leave some low priority processes waiting indefinitely.

• In a heavily loaded computer system, a steady stream of higher-priority processes can prevent a low-priority process from ever getting the CPU.

Solution to the Problem:

• A solution to the problem of indefinite blockage of low-priority processes is aging.

• Aging is a technique of gradually increasing the priority of processes that wait in the system for a long time.

• For example, if priorities range from 127 (low) to 0 (high), we could increase the priority of a waiting process by 1 every 15 minutes.

• Eventually, even a process with the initial priority of 127 would have a highest priority in the system and would be executed.

Problems 1:

Consider the set of processes with arrival time (in milliseconds), CPU burst time (in milliseconds), and priority (0 is the highest priority) shown below. None of the processes have I/O burst time.

Process ID	Arrival Time	Burst Time	Priority
P1	0	11	2
P2	5	28	0
P3	12	2	3
P4	2	10	1
P5	9	16	4

The average waiting time (in milliseconds) of all the processes using pre-emptive priority scheduling algorithm is__29__.

Solution:

Process ID	Arrival Time	Burst Time	Priority
P1	0	~~11~~ 9	2
P2	5	28	0
P3	12	2	3
P4	2	~~10~~ 7	1
P5	9	16	4

Gantt Chart:

P1	P4	P2	P4	P1	P3	P5

0 2 5 33 40 49 51 67

Waiting Time = Total Waiting time -No. of ms process executed -Arrival Time

Waiting Time for P1 = (40 - 2 - 0) = 38 ms

Waiting Time for P2 = (5 - 0 - 5) = 0 ms

Waiting Time for P3 = (49 - 0 - 12) = 37 ms

Waiting Time for P4 = (33 - 3 - 2) = 28 ms

Waiting Time for P5 = (51 - 0 - 9) = 42 ms

Average Waiting Time

= (38 + 0 + 37 + 28 + 42) / 5

= 145/5 ms

= 29 ms

Problems 2:

Consider the set of processes with arrival time (in milliseconds), CPU burst time (in milliseconds), and priority shown below: (Higher number represents higher priority)

Process ID	Arrival Time	Burst Time	Priority
P1	0	4	2
P2	1	3	3
P3	2	1	4
P4	3	5	5
P5	4	2	5

If the CPU scheduling policy is priority non-pre-emptive, calculate the average waiting time and average turnaround time.

Solution:

Gantt Chart:

P1	P4	P5	P3	P2

0 4 9 11 12 15

Turnaround Time = Exit Time - Arrival Time

Waiting Time = Turnaround Time - Burst Time

Process ID	Arrival Time	Burst Time	Priority	Exit Time	Turnaround time	Waiting Time
P1	0	4	2	4	4	0
P2	1	3	3	15	14	11
P3	2	1	4	12	10	9
P4	3	5	5	9	6	1
P5	4	2	5	11	7	5

Average Turn Around time = (4 + 14 + 10 + 6 + 7) / 5

= 41 / 5

= 8.2 ms

Average waiting time = (0 + 11 + 9 + 1 + 5) / 5

= 26 / 5

= 5.2 ms

Round-Robin Scheduling

• The round-robin (RR) scheduling algorithm is designed especially for timesharing systems.

• It is similar to FCFS scheduling, but pre-emption is added to switch between processes.

• A small unit of time, called a time quantum or time slice, is defined (generally from 10 to 100 milliseconds)

• The ready queue is treated as a circular queue.

The CPU scheduler goes around the ready queue, allocating the CPU to each process for a time interval of up to 1 time quantum.

Implementation of Round Robin scheduling:

• We keep the ready queue as a FIFO queue of processes.

• New processes are added to the tail of the ready queue.

• The CPU scheduler picks the first process from the ready queue, sets a timer to interrupt after 1 time quantum, and dispatches the process.

One of two things will then happen

- The process may have a CPU burst of less than 1 time quantum
  - The process itself will release the CPU voluntarily
  - The CPU scheduler will then proceed to the next process in the ready queue
- The CPU burst of the currently running process is longer than 1 time quantum, the timer will go off and will cause an interrupt to the O.S.
  - A context switch will be executed, and the process will be put at the tail of the ready queue
  - The CPU scheduler will then select the next process in the read queue

Turnaround Time and Waiting Time (Round-Robin Scheduling)

Consider the following set of processes that arrive at time 0, with the length of the CPU burst given in milliseconds and time quantum taken as 4 milliseconds for RR Scheduling:

Process ID	Burst Time
P1	24
P2	3
P3	3

Gantt Chart:

P1	P2	P3	P1	P1	P1	P1	P1

0 4 7 10 14 18 22 26 30

Method 1:

Turnaround Time = Exit Time - Arrival Time

Waiting Time = Turnaround Time - Burst Time

Process ID	Exit Time	Turnaround Time	Waiting Time
P1	30	30	6
P2	7	7	4
P3	10	10	7

Average Turn Around time = (30 + 7 + 10) / 3

= 47 / 3

= 15.66 ms

Average waiting time = (6 + 4 + 7) / 3

= 17 / 3

= 5.66 ms

Method 2:

Waiting Time = Last Start Time - Arrival Time – (Preemption x Time Quantum)

Process ID	Waiting Time
P1	26 – 0 – (5x4) = 6
P2	4 – 0 – (0x4) = 4
P3	7 – 0 – (0x4) = 7

Average waiting time = (6 + 4 + 7) / 3

= 17 / 3

= 5.66 ms

Problem 1:

https://youtu.be/QlCmgBOMjlI?si=usug2VMe-4MkxBaC

Problem 2:

https://youtu.be/wioTortHb_g?si=pf9jEg1yUw-5r3xD